

Randomized Algorithms: Las Vegas and Monte Carlo

Programming and Data Structures — Week 14, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to distinguish a Las Vegas algorithm from a Monte Carlo algorithm precisely, using Week 6's randomized quicksort as the anchor Las Vegas example; explain why randomized quicksort's expected running time is $O(n \log n)$, connecting directly back to Week 6 Lecture 2's worst-case analysis; implement and trace a genuine Monte Carlo algorithm, randomized primality testing, and explain its error probability precisely; and state which correctness guarantee each algorithm type actually provides, explaining why conflating the two is a real and serious mistake.

Outline

Today covers a two-kinds-of-gambling analogy distinguishing randomized algorithm categories; Las Vegas algorithms, anchored in Week 6's randomized quicksort; why “expected” time is a genuinely different guarantee than “worst-case” time; Monte Carlo algorithms, using randomized primality testing as a complete worked example; the critical distinction between always-correct-but-variable-time and always-fast- but-possibly-wrong algorithms; why conflating the two categories is a real and serious mistake; the MTech-track technique of amplification, reducing Monte Carlo error probability toward zero; an in-class quiz; and a summary.

The two-kinds-of-gambling analogy

These two algorithm categories borrow their names from two styles of gambling, used here as a memory aid, not a literal description. A “Las Vegas” algorithm might take longer than expected on an unlucky run, but is guaranteed to eventually produce the correct answer. A “Monte Carlo” algorithm produces an answer quickly, on a fixed, predictable schedule, but carries a small, precisely quantifiable chance that the answer returned is actually wrong. Both categories use randomness deliberately — the essential difference is exactly what the randomness is permitted to affect: running time, in the Las Vegas case, or correctness itself, in the Monte Carlo case.

Las Vegas algorithms: randomized quicksort, revisited

Week 6 Lecture 2's randomized-pivot quicksort is the canonical Las Vegas algorithm already built in this course. It always produces a correctly sorted array, on every single run, with no exceptions whatsoever — correctness is never in question. What the randomness affects is exclusively how long the algorithm takes: its expected running time is $O(n \log n)$, but a worst case of $O(n^2)$, while extremely unlikely with random pivot selection, remains theoretically possible on any given run, exactly Week 6 Lecture 2's own honest caveat.

Why “expected” time is a genuinely different guarantee

It is worth being precise about a distinction easy to blur. A worst-case bound of $O(n \log n)$, as merge sort provides, holds on every single run without exception, guaranteed. An expected bound of $O(n \log n)$, as randomized quicksort provides, is an average taken over the algorithm's internal randomness — most runs land near this bound, but any individual run could, in principle, be considerably worse, even though this becomes vanishingly unlikely as n grows. Confusing “expected” with “worst-case” is a real and serious error, which is exactly why Week 6 Lecture 2 was careful to distinguish these two claims explicitly rather than treating them as interchangeable.

Monte Carlo algorithms: randomized primality testing

```
import random

def miller_rabin_witness(n, a):
    # returns True if 'a' proves n is COMPOSITE (definitely not prime)
    # returns False if 'a' provides no such proof (n MIGHT be prime)
    if n % a == 0:
        return True
    d = n - 1
    while d % 2 == 0:
        d //= 2
    x = pow(a, d, n)
    if x == 1 or x == n - 1:
        return False
    while d != n - 1:
        x = pow(x, 2, n)
        d *= 2
        if x == n - 1:
            return False
    return True
```

```
def is_probably_prime(n, k=20):
    if n < 2:
        return False
    for _ in range(k):
        a = random.randint(2, n - 1)
        if miller_rabin_witness(n, a):
            return False          # definitely composite
    return True                  # PROBABLY prime - not guaranteed
```

The Miller-Rabin primality test is a genuine Monte Carlo algorithm: it runs in polynomial time, dramatically faster than checking every possible divisor, and completes in a fixed number of rounds, k , regardless of the specific input. If it reports a number as composite, that answer is always correct with absolute certainty — a witness value proving compositeness is a mathematical proof, not a probabilistic claim. If it reports a number as “probably prime,” however, there remains a small, precisely quantifiable chance the answer is wrong — a genuinely composite number that happened, by bad luck, to fool every one of the k randomly chosen witness values.

Tracing Miller-Rabin on a small example

Testing $n=221$ ($= 13 \times 17$, genuinely composite) with witness $a=174$:

```
d = 220, reduce by 2s: 220 -> 110 -> 55 (odd, stop). d=55
x = 174^55 mod 221 = 47 (computed via modular exponentiation)
47 != 1 and 47 != 220 (n-1) -> continue the squaring loop
square: 47^2 mod 221 = 220 = n-1 -> return False (a=174 does NOT prove composite)
```

Testing the SAME $n=221$ with witness $a=137$:

```
d=55. x = 137^55 mod 221 = 188
188 != 1, 188 != 220 -> square: 188^2 mod 221 = 205, not 220 either
d reaches n-1 without ever hitting n-1 -> return True (137 PROVES 221 is composite)
```

Tracing Miller-Rabin on $n = 221$, which is genuinely composite (13×17): witness $a = 174$ happens not to expose the compositeness, incorrectly suggesting (from this one witness alone) that 221 might be prime. Witness $a = 137$, however, does correctly expose it as composite. This illustrates precisely why the algorithm uses k independent random witnesses rather than just one — a single unlucky witness can fail to detect compositeness, but requiring many independent witnesses to all fail simultaneously is what drives the error probability down to a controllable, quantifiable level.

Verifying `is_probably_prime` against known values

```
for n in [17, 97, 7919]:
    print(n, is_probably_prime(n))    # all True - genuine primes

for n in [221, 1000, 8051]:
    print(n, is_probably_prime(n))    # all False - genuine composites, caught with certainty
```

Running `is_probably_prime` on three known primes (17, 97, and 7919) correctly reports `True` for all three, and on three known composites (221, 1000, and 8051) correctly reports `False` for all three — every composite tested was caught with the $k=20$ rounds used, exactly as expected given the astronomically small error probability this lecture’s MTech addition quantifies precisely.

The critical distinction, stated plainly

This table states the critical distinction plainly, in a form worth memorizing precisely rather than approximately. Las Vegas algorithms, anchored by randomized quicksort, are always correct, with randomness affecting only running time, which varies and only has an expected, not worst-case, bound. Monte Carlo algorithms, anchored by Miller-Rabin, run in fixed, predictable time, but carry a genuine, quantifiable chance of being incorrect, with randomness affecting correctness itself, not merely speed.

Why conflating the two is a real, serious mistake

Deploying a Monte Carlo algorithm in a setting that actually requires an always-correct guarantee is a genuine, real-world risk, not merely a theoretical concern — cryptographic key generation relying on a “probably prime” number that happens to actually be composite is a genuine, exploitable security vulnerability, which is why real cryptographic systems use a sufficiently large number of rounds k to make this probability astronomically small, addressed in this lecture’s MTech section. Deploying a Las Vegas algorithm’s expected-time bound as if it were a worst-case guarantee, in a hard-real-time system such as a medical device or flight control software, risks a genuinely missed deadline with real consequences. Both mistakes share an identical root cause: failing to check which specific guarantee an algorithm actually provides before relying on it in a consequential setting.

MTech addition — amplification: reducing error probability

Miller-Rabin’s error probability for a single round is at most $1/4$, a proven mathematical bound — genuinely too high to trust on its own for any consequential use. Running k independent rounds, all agreeing on “probably prime,” reduces the overall error probability to at most $(1/4)^k$, since all k

independent rounds would need to simultaneously fail to catch the compositeness. For $k = 20$, as used in this lecture's code, the error probability is at most $(1/4)^{20} \approx 10^{-12}$ — astronomically small, in practice indistinguishable from certainty for any real application, though it is important to state precisely that this is not mathematically zero, a genuine and honest distinction from a true correctness guarantee.

Why Miller-Rabin is fast: connecting back to Week 1

Checking every possible divisor up to $\sqrt{n}$, the naive primality test, costs $O(\sqrt{n})$ — but for a number with hundreds of digits, as used in real cryptography, $\sqrt{n}$ itself is astronomically large, making this approach genuinely infeasible, not merely slow. Miller-Rabin's modular exponentiation, computed via `pow(a, d, n)` using repeated squaring, costs only $O(\log n)$ per operation — exactly Week 1 Lecture 4's logarithmic-growth idea, applied here to modular arithmetic rather than search. This is precisely why accepting a small, quantifiable error risk is worth it: the fully deterministic alternative is not simply slower at cryptographic scale, it is entirely infeasible.

Exercise

As an exercise, implement `is_probably_prime` exactly as presented in this lecture, and verify it correctly identifies known prime numbers (17, 97, 7919) as “probably prime” and known composite numbers (221, 1000, 8051) as definitely composite. Separately, compute by hand the error probability bound $(1/4)^k$ for $k = 1, 5, 10$, and 20 rounds, observing directly how quickly the bound becomes practically negligible.

A second worked Miller-Rabin trace: $n = 561$

```
n, d, r = 561, 35, 4
x = pow(2, d, n)
print(x)          # 263 - not 1, not 560
for _ in range(r - 1):
    x = pow(x, 2, n)
    print(x)       # 166, 67, 1 -- hits 1 without passing through 560 first
```

The value 561 equals $3 \times 11 \times 17$ and is a Carmichael number — notoriously the smallest composite number that fools the older, weaker Fermat primality test for every possible base. Writing $560 = 2^4 \times 35$ gives $d = 35$ and $r = 4$. Testing witness $a = 2$ computes $2^{35} \bmod 561 = 263$, which is neither 1 nor 560, so the squaring loop continues: 263 squares to 166, then 67, then finally 1 — but critically, 1 was reached *without* the sequence passing through 560 (the value $n - 1$) immediately beforehand. That specific pattern is exactly the composite-witness condition Miller-Rabin is built to

detect, and running it confirms $a = 2$ correctly identifies 561 as composite even though the older Fermat test would have missed it.

Why Miller-Rabin succeeds where Fermat's test fails on Carmichael numbers

Fermat's test only checks whether $a^{n-1} \equiv 1 \pmod{n}$, and Carmichael numbers are specifically constructed to satisfy exactly that congruence for every base a coprime to n — which is precisely what makes them fool Fermat's test unconditionally. Miller-Rabin strengthens the check by inspecting every intermediate square root encountered while computing that final value of 1 through repeated squaring. In a true prime, modular arithmetic guarantees the only square roots of 1 are $+1$ and -1 . If the repeated-squaring sequence ever produces a square root of 1 that is neither of those two values, n is definitely composite — this extra structural check is exactly what a Carmichael number's disguise cannot survive, and it is why Miller-Rabin is preferred over the older, weaker Fermat test in real cryptographic libraries.

Real-world use of randomized algorithms

RSA and other public-key cryptographic systems generate the large prime numbers they depend on using Miller-Rabin or closely related probabilistic primality tests directly — this lecture's algorithm is not a classroom simplification but genuinely how real cryptographic key generation works in practice. Randomized quicksort, covered in Week 6, is used in real standard library sort implementations specifically to avoid the adversarial worst-case inputs a deterministic pivot rule would be vulnerable to. Hash table security, touched on in Week 8, randomizes the hash function's internal seed at program startup specifically to defend against deliberately crafted “hash flooding” denial-of-service attacks — a genuine security application of randomization defending against an adversary who knows the algorithm but not the specific random values chosen at runtime.

Quiz — categorize the algorithm

Given an algorithm that always returns the correct answer, but whose running time ranges from a fraction of a second to, extremely rarely, several minutes depending on random internal choices, determine before checking the answer whether this is a Las Vegas or Monte Carlo algorithm.

Quiz — answer

This is a Las Vegas algorithm. Correctness is guaranteed on every single run — the answer is always correct — and only the running time varies depending on the algorithm's internal random choices. This is exactly randomized quicksort's own profile, described in different words in this quiz: always correct, expected fast, with a rare, unlikely worst case remaining theoretically possible.

Connecting back: randomization across the whole course

Week 6's randomized quicksort is a Las Vegas algorithm, and this week's Miller-Rabin primality test is a Monte Carlo algorithm — two structurally different randomization strategies solving two very different problems, sorting versus primality testing. Week 8's hash table seed randomization supplies a third example, using randomness not primarily for speed or feasibility but as a security defense against an adversary who knows the algorithm but not its runtime random choices. The unifying lesson across all three is that randomness is a legitimate, rigorous design tool in computer science, provided its exact guarantee — always-correct-but-variable-running-time for Las Vegas, fast-but-carrying-a-tiny-controllable-error-chance for Monte Carlo — is stated and understood with full precision rather than treated as a vague hand-wave.

Summary

Las Vegas algorithms, anchored by Week 6's randomized quicksort, are always correct, with only running time varying and only an expected, not worst-case, bound available. Monte Carlo algorithms, anchored by Miller-Rabin primality testing, run in fixed, predictable time but carry a small, precisely quantifiable chance of being wrong. Conflating “expected” with “worst-case,” or deploying a Monte Carlo algorithm in a setting that genuinely requires certainty, are both real and serious mistakes with real consequences, not merely academic technicalities. Amplification, running multiple independent rounds, drives Monte Carlo error probability toward zero, though never mathematically to exactly zero — an honest distinction worth carrying forward. Next lecture closes this course, consolidating the entire semester ahead of the closing project-presentation week.